

CSC384
Knowledge Representation & Reasoning
Part 3

Bahar Aameri

We gratefully acknowledge those who have contributed to these slides, most recently Bahar Aameri, who merged and augmented slides from Yongmei Liu and a CSC384 slide deck historically developed by Craig Boutilier, Fahiem Bacchus, Sheila McIlraith, Sonya Allin, Hojjat Ghaderi, and others. We also acknowledge the use of material written by Michael Winter, and the use of material originating from slides and the book by Ron Brachman and Hector Levesque.

Logical Consequence

Let Φ be a set of sentences and A be a sentence.

A is a **logical consequence** of Φ (denoted by $\Phi \models A$) iff for every structure $\mathcal{M}$, if $\mathcal{M} \models \Phi$ then $\mathcal{M} \models A$.

$\mathcal{M}_1 \models A$

If A is a logical consequence of Φ , then there is no $\mathcal{M}$ such that $\mathcal{M} \models \Phi \cup \{\neg A\}$.
In other words, $\Phi \cup \{\neg A\}$ is **unsatisfiable**.

Example:

Assume Φ includes the following sentences:

$\forall x \forall y \forall z [(above(z, y) \wedge above(y, x)) \rightarrow above(z, x)]$

$above(c_1, c_2) \wedge above(c_2, c_3)$

} $above(c_1, c_3)$

Knowledge-based Systems

Knowledge Base (KB): A collection of **sentences** that represents what the **agent/program believes** about the world.

Sentences in the KB are **explicit** knowledge of the agent.

Logical consequences of the KB are **implicit** knowledge of the agent.

Example: Suppose **KB** includes the following sentences:

- The capital of Canada is Ottawa
- The largest province in Canada is Quebec
- The provinces neighbouring Quebec are Ontario, New Brunswick, and Newfoundland

Implicit knowledge of the KB:

Ontario, New Brunswick and Newfoundland are the neighbouring provinces of the largest province in Canada.

- To compute implicit knowledge of the KB (i.e., logical consequences) we need a **mechanical procedure** that can be implemented as an **algorithm**.
- This would allow us to **reason** with our **knowledge**:
 - Represent the knowledge as **logical formulas**.
 - Apply the **procedure** for generating logical consequences
- Mechanical proof procedures work by **manipulating formulas**. They do not know or care anything about interpretations. Nevertheless they **respect the semantics** of interpretations!

Proof Procedures

A proof procedure is **sound** if whenever it **produces** a sentence A by manipulating sentences in a KB, then A is a **logical consequence** of KB (i.e., $KB \models A$).

That is, **all conclusions** arrived at via the proof procedure are **correct**: they are logical consequences.

A proof procedure is **complete** if it can produce **all logical consequences** of KB.

That is, if $KB \models A$, then the procedure can **produce** A .

Resolution by Refutation

Resolution works with formulas expressed in **clausal form**.

A **literal** is an **atomic formula** or the **negation** of an atomic formula.

Example: $\text{dog}(\text{fido})$, $\neg \text{cat}(\text{fido})$, $P(x)$, $\neg Q(y)$

A **clause** is a **disjunction** of literals:

Example:

$P(x) \vee \neg Q(x, y)$

$\neg \text{Owns}(\text{fido}, \text{fred}) \vee \neg \text{Dog}(\text{fido}) \vee \text{Person}(\text{fred})$

$T \leftarrow Q \vee P \rightarrow T \quad F$
 $T \leftarrow R \vee \neg P \rightarrow F \quad T$

A **clausal theory** is a **set of clauses**.

It can also be considered as **conjunction** of clauses.

Example:

$\{P(x) \vee \neg Q(x, y), \neg \text{Owns}(\text{fido}, \text{fred}) \vee \neg \text{Dog}(\text{fido}) \vee \text{Person}(\text{fred})\}$

$R \vee Q$

Resolution proof procedure uses the following **inference rule**:

$$\frac{a_1 \vee a_2 \vee \dots \vee a_n \vee \underline{c} \quad b_1 \vee b_2 \vee \dots \vee b_m \vee \underline{\neg c}}{a_1 \vee a_2 \vee \dots \vee a_n \vee b_1 \vee b_2 \vee \dots \vee b_m}$$

Resolution by Refutation

$(Q(x, y) \vee \underline{P(a)})$ and $(R(y) \vee \underline{\neg P(a)})$

$Q(x, y) \vee R(y)$

$(\underline{Q(x, y) \vee P(a)})$ and $\neg P(a)$

$Q(x, y)$

$P(a)$ and $\neg P(a)$

$()$

We denote a **contradiction** by an **empty clause**: $()$

$KB \cup \{\neg A\}$

Resolution by Refutation to show $KB \models A$:

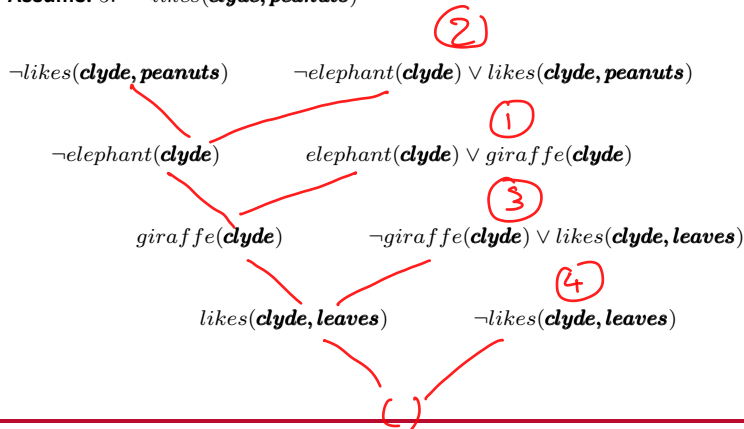
- Assume $\neg A$ is true to generate a contradiction. (**Refutation**)
- Convert $\neg A$ and all sentences in KB to a **clausal theory** C .
- **Resolve** the clauses in C until an **empty clause** is obtained.

Resolution by Refutation: Example

Want to prove likes(clyde,peanuts) from:

1. $\text{elephant}(\text{clyde}) \vee \text{giraffe}(\text{clyde})$
2. $\neg \text{elephant}(\text{clyde}) \vee \text{likes}(\text{clyde}, \text{peanuts})$ ← KB
3. $\neg \text{giraffe}(\text{clyde}) \vee \text{likes}(\text{clyde}, \text{leaves})$ ←
4. $\neg \text{likes}(\text{clyde}, \text{leaves})$

Assume: 5. $\neg \text{likes}(\text{clyde}, \text{peanuts})$



Resolution by Refutation: Example

Want to prove $\text{likes}(\text{clyde}, \text{peanuts})$ from:

1. $\text{elephant}(\text{clyde}) \vee \text{giraffe}(\text{clyde})$
2. $\neg \text{elephant}(\text{clyde}) \vee \text{likes}(\text{clyde}, \text{peanuts})$
3. $\neg \text{giraffe}(\text{clyde}) \vee \text{likes}(\text{clyde}, \text{leaves})$
4. $\neg \text{likes}(\text{clyde}, \text{leaves})$

Resolution by Refutation Proof:

- $\neg \text{likes}(\text{clyde}, \text{peanuts})$ [5.]
- 5&2: $\neg \text{elephant}(\text{clyde})$ [6.]
- 6&1: $\text{giraffe}(\text{clyde})$ [7.]
- 7&3: $\text{likes}(\text{clyde}, \text{leaves})$ [8.]
- 8&4: ()

To develop a complete resolution proof procedure for first-order logic we need :

1. A way of **converting** KB and A into **clausal form**. ✓
2. A way of doing **resolution** even when literals are **not syntactically identical** (**unification**). ←

↪ optional

Conversion to Clausal Form

1. Eliminate Implications.
2. Move Negations Inwards (and simplify $\neg\neg$).
3. Standardize Variables.
4. Skolemization.
5. Convert to Prenex Form.
6. Distribute Conjunctions over Disjunctions.
7. Flatten nested Conjunctions and Disjunctions.
8. Convert to Clauses.

$$(P_1 \vee P_2 \vee \dots \vee P_n) \wedge (Q_1 \vee Q_2 \vee \dots \vee Q_m)$$

Eliminate Implications

Implication Rule: $A \rightarrow B$ iff $\neg A \vee B$

$$\forall x \left[\underline{P(x)} \rightarrow \left(\left(\forall y [\underline{P(y)} \rightarrow P(f(x, y))] \right) \wedge \neg \left(\forall y [\neg q(x, y) \wedge P(y)] \right) \right) \right]$$

Eliminate Implication: $\forall x \left[\underline{\neg P(x)} \vee \left(\left(\forall y [\underline{\neg P(y)} \vee P(f(x, y))] \right) \wedge \neg \left(\forall y [\neg q(x, y) \wedge P(y)] \right) \right) \right]$

Rules for Simplifying and Moving Negations Inwards

- $\neg\neg A$ iff A



- $\neg(A \wedge B)$ iff $\neg A \vee \neg B$
- $\neg(A \vee B)$ iff $\neg A \wedge \neg B$

- $\neg\forall x A$ iff $\exists x \neg A$

- $\neg\exists x A$ iff $\forall x \neg A$

Simplify and Move Negations Inwards

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge \neg (\forall y [\neg Q(x, y) \wedge P(y)]) \right) \right]$$

Move Negations Inwards:

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (\exists y [\neg \neg Q(x, y) \vee \neg P(y)]) \right) \right]$$

Simplify Negations:

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (\exists y [Q(x, y) \vee \neg P(y)]) \right) \right]$$

Standardize Variables

Standardize Variables: Rename variables so that each quantified variable is **unique**.

$$\forall x \left[\neg P(x) \vee \left(\underbrace{(\forall y [\neg P(y) \vee P(f(x, y))])}_{\text{blue bracket}} \wedge \underbrace{(\exists y [Q(x, y) \vee \neg P(y)])}_{\text{blue bracket}} \right) \right]$$

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (\exists z [Q(x, z) \vee \neg P(z)]) \right) \right]$$

Skolemization: Remove existential quantifiers by introducing new function symbols.

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (\exists z [Q(x, z) \vee \neg P(z)]) \right) \right]$$

- Consider $\exists y(\text{elephant}(y) \wedge \text{friendly}(y))$
- This asserts that there is **some individual** (binding for y) that is both an elephant and friendly.
- To remove the existential, we invent a "name" for this individual a .
This "name" must be a **new constant symbol** (not equal to any previous constant symbols in the vocabulary of the KB):

$$\text{elephant}(a) \wedge \text{friendly}(a)$$

Skolemization

- Consider $\exists y(\text{elephant}(y) \wedge \text{friendly}(y))$
- This asserts that there is **some individual** (binding for y) that is both an elephant and friendly.
- To remove the existential, we invent a "name" for this individual a .
This "name" must be a **new constant symbol** (not equal to any previous constant symbols in the vocabulary of the KB):

$$\text{elephant}(a) \wedge \text{friendly}(a)$$

- The new sentence says **the same thing**, since we do not know anything about a .
- **IMPORTANT:** The introduced symbol a must be **new**.
Else we might **know something else** about a in KB.
 - If we did know something else about a we would be asserting **more than the existential**.
 - In original quantified formula we know nothing about the variable y . Just what was being asserted by the existential formula.

- Now consider

$$\forall x \exists y (\text{loves}(x, y))$$

This formula states that for **every** x there is **some** y that x loves (possibly a different y for each x).

- Replacing the existential by a new constant **won't work**

$$\forall x (\text{loves}(x, \mathbf{a}))$$

This asserts that there is a **particular individual** $\mathbf{a}$ loved by **every** x .

Skolemization

- Now consider

$$\forall x \forall z \exists y (\text{loves}(x, y)) \quad \rightarrow \quad g(x, z, x)$$

This formula states that for **every** x there is **some** y that x loves (possibly a different y for each x).

- Replacing the existential by a new constant **won't work**

$$\forall x (\text{loves}(x, a))$$

This asserts that there is a **particular individual** a loved by **every** x .

- To properly convert existential quantifiers **scoped by universal** quantifiers we must use **functions**:
 - Use a **new function symbol** that mentions **every universally quantified variable** that **scopes the existential**.

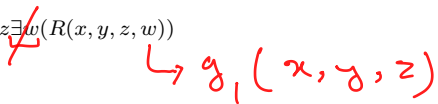
$$\forall x (\text{loves}(x, \underline{g(x)}))$$

where g is a **new** function symbol.

This formula asserts that for every x there is some individual (denoted by $g(x)$) that x loves.

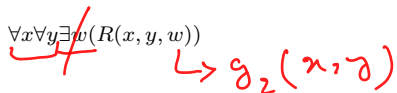
Skolemization: Examples

$$\forall x \forall y \forall z \exists w (R(x, y, z, w))$$



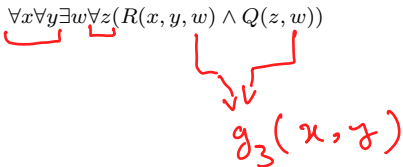
$$\hookrightarrow g_1(x, y, z)$$

$$\forall x \forall y \exists w (R(x, y, w))$$



$$\hookrightarrow g_2(x, y)$$

$$\forall x \forall y \exists w \forall z (R(x, y, w) \wedge Q(z, w))$$



$$\hookrightarrow g_3(x, y)$$

Skolemization: Remove existential quantifiers by introducing new function symbols.

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (\exists z [Q(x, z) \vee \neg P(z)]) \right) \right]$$

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (Q(x, g(x)) \vee \neg P(g(x))) \right) \right]$$

Convert to Prenex Form

Convert to Prenex Form: Bring all **quantifiers** to the **front**.

We use the following equivalences, where x does not occur free in Q

$$\bullet \quad \forall x P \wedge Q \quad \text{iff} \quad Q \wedge \forall x P \quad \text{iff} \quad \forall x (P \wedge Q)$$

$$\bullet \quad \forall x P \vee Q \quad \text{iff} \quad Q \vee \forall x P \quad \text{iff} \quad \forall x (P \vee Q)$$

$$\forall x \left[\neg P(x) \vee \left((\forall y [\neg P(y) \vee P(f(x, y))]) \wedge (Q(x, g(x)) \vee \neg P(g(x))) \right) \right]$$

$$\forall x \forall y \left[\neg P(x) \vee \left((\neg P(y) \vee P(f(x, y))) \wedge (Q(x, g(x)) \vee \neg P(g(x))) \right) \right]$$

Distribute Conjunctions over Disjunctions

Conjunctions over Disjunctions: $A \vee (B \wedge C) \quad \text{iff} \quad (A \vee B) \wedge (A \vee C)$

$$\forall x \forall y \left[\neg P(x) \vee \left((\neg P(y) \vee P(f(x, y))) \wedge (Q(x, g(x)) \vee \neg P(g(x))) \right) \right]$$

$$\forall x \forall y \left[\left(\neg P(x) \vee (\neg P(y) \vee P(f(x, y))) \right) \wedge \left(\neg P(x) \vee (Q(x, g(x)) \vee \neg P(g(x))) \right) \right]$$

Flatten nested Conjunctions and Disjunctions

Flatten nested $\wedge$ and $\vee$:

- $A \vee (B \vee C)$ to $(A \vee B \vee C)$
- $A \wedge (B \wedge C)$ to $(A \wedge B \wedge C)$

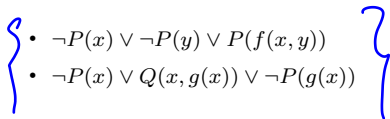
$$\forall x \forall y \left[\left(\neg P(x) \vee (\neg P(y) \vee P(f(x, y))) \right) \wedge \left(\neg P(x) \vee (Q(x, g(x)) \vee \neg P(g(x))) \right) \right]$$

$$\forall x \forall y \left[\left(\neg P(x) \vee \neg P(y) \vee P(f(x, y)) \right) \wedge \left(\neg P(x) \vee Q(x, g(x)) \vee \neg P(g(x)) \right) \right]$$

Convert to Clauses

Convert to Clauses: Remove universal quantifiers and break apart conjunctions

$$\forall x \forall y \left[\left(\neg P(x) \vee \neg P(y) \vee P(f(x, y)) \right) \wedge \left(\neg P(x) \vee Q(x, g(x)) \vee \neg P(g(x)) \right) \right]$$

- 
- $\neg P(x) \vee \neg P(y) \vee P(f(x, y))$
 - $\neg P(x) \vee Q(x, g(x)) \vee \neg P(g(x))$

- If clauses are **syntactically identical**, we can easily detect if a P and $\neg P$ exists.
- What about when they're not? Can the following clauses be resolved?

$(P(\text{john}), Q(\text{fred}), R(x))$

$(\neg P(y), R(\text{susan}), R(y))$

- Once reduced to clausal form, all remaining **variables** are **universally quantified**.
So, implicitly $(\neg P(y), R(\text{susan}), R(y))$ represents a whole set of clauses like
 $(\neg P(\text{fred}), R(\text{susan}), R(\text{fred}))$
 $(\neg P(\text{john}), R(\text{susan}), R(\text{john}))$
...
- So there is a **specialization** of this clause that can be resolved with
 $(P(\text{john}), Q(\text{fred}), R(x))$
- In particular
 $(P(\text{john}), Q(\text{fred}), R(\text{john}))$ and $(\neg P(\text{john}), R(\text{susan}), R(\text{john}))$
can be resolved, producing
 $(Q(\text{fred}), R(\text{john}), R(\text{susan}))$

Unification

- We want to be able to **match conflicting literals**, even when they are **not syntactically identical**.
- The matching process **automatically** determines whether or not there is a **specialization that matches**.
- But, We don't want to **over specialize!**

- ($\neg P(x)$, $S(x)$, $Q(\text{fred})$)

- ($P(y)$, $R(y)$)

Possible resolvents:

$(S(\text{john}), Q(\text{fred}), R(\text{john}))\{y = x, x = \text{john}\}$

$(S(\text{sally}), Q(\text{fred}), R(\text{sally}))\{y = x, x = \text{sally}\}$

$(S(x), Q(\text{fred}), R(x))\{y = x\}$

- The last resolvent is **most-general**, the other two are specializations of it.
We want to keep the most general clause so that we can use it in future resolution steps.

See appendix for details on Unification and finding the Most General Unifier (Optional!).

Concluding Remarks: Decidability of FOL

There can be no procedure that does this:

Proc(*clauses*):

 if *clauses* are unsatisfiable:

 return YES

 else:

 return NO

- Resolution is *refutation complete*.

If a set of clauses is unsatisfiable (i.e., when the answer is "YES") and so some branch contains [], a *breadth-first* search guaranteed to find [].

- But search may not terminate on *satisfiable* clauses (i.e., when the answer is "NO").

Concluding Remarks: Decidability of FOL

In general,

- first-order unsatisfiability is semi-decidable, but not decidable.
Thus, calculating entailments is semi-decidable and undecidable.
- first-order satisfiability is undecidable.

Loosely speaking, a decision problem is

- *decidable* if there is some algorithm that correctly generates a "YES-NO" answer for every possible input.
Otherwise, it's *undecidable*.
- *semi-decidable* if there is some algorithm that correctly generates "YES" answers, but does not terminate on some inputs for which the answer is "NO".

Concluding Remarks – Implications for KR&R

- Even though unsatisfiability is semi-decidable, generating YES-answer might take a *long time*.
- In some applications, it is reasonable to wait.
Example: Mathematical theorem proving, where the user care about proving specific formulas.
- **Automated Theorem Proving:** an area of AI that studies strategies for automatically proving difficult theorems.
- But, the exiting theorem-provers are (still) *not good enough* for most KR&R applications (because we usually want an immediate answer).

Problem:

Want to *produce entailments* of KB as needed for *immediate* action.

But full theorem-proving may be *too difficult for KR&R!*

Need to consider *other options ...*

Problem:

Want to *produce entailments* of KB as needed for *immediate* action.

But full theorem-proving may be *too difficult* for KR&R!

Need to consider *other options* ...

Possible Solutions:

- **Satisfiability:** Some first-order cases can be handled by converting them to a *propositional form*.
- **Calculating Entailment (Unsatisfiability):**
 - Giving *control to user*.
Example: Procedural Control of Reasoning.
 - Using *decidable fragments* of FOL (which are *less expressive*).
Example: Description Logics, Horn Clauses.

Appendix:

Unification and Finding Most General Unifier (Optional Material)

Substitution

- **Unification** is a mechanism for finding the **most general matching**.
- A key component of unification is **substitution**.
A **substitution** is a finite **set of equations** of the form $V = t$ where V is a variable and t is a term not containing V (t might contain other variables).
- We can apply a substitution $\delta = \{V_1 = t_1, \dots, V_n = t_n\}$ to a formula A to obtain a new formula $A\delta$ by **simultaneously** replacing every variable V_i by term t_i .

Example: Applying $\delta = \{x = y, y = f(a)\}$ to $P(x, g(y, z))$

$$P(x, g(y, z))\{x = y, y = f(a)\} \Rightarrow P(y, g(f(a), z))$$

Note that the substitutions are **NOT** applied **sequentially**, i.e., the first y is not subsequently replaced by $f(a)$.

Composition of Substitutions

- We can **compose** two substitutions θ and δ to obtain a **new** substitution $\theta\delta$.
- **Composition** is a way of converting the **sequential application** of a series of substitutions to a **single simultaneous** substitution.

$$\theta = \{x_1 = s_1, x_2 = s_2, \dots, x_m = s_m\}$$

$$\delta = \{y_1 = t_1, y_2 = t_2, \dots, y_k = t_k\}$$

To compute $\theta\delta$:

1. Apply δ to each RHS of θ and then add all of the equations of δ :

$$\theta\delta = \{x_1 = s_1\delta, x_2 = s_2\delta, \dots, x_m = s_m\delta, y_1 = t_1, y_2 = t_2, \dots, y_k = t_k\}$$

2. Delete any identities, i.e., equations of the form $V = V$ from $\theta\delta$.
3. Delete any equation $y_i = s_i$ where y_i is equal to one of the x_j in θ .

Example: $\theta = \{x = f(y), y = z\}$, $\delta = \{x = a, y = b, z = y\}$

$$\begin{aligned}\theta\delta: S &= \{x = f(y)\{x = a, y = b, z = y\}, y = z\{x = a, y = b, z = y\}, x = a, y = b, z = y\} \\ &= \{x = f(b), y = y, x = a, y = b, z = y\}\end{aligned}$$

$\{x = f(b), x = a, y = b, z = y\}$ delete identities

$\{x = f(b), z = y\}$ x and y are in θ

$$\theta\delta = \{x = f(b), z = y\}$$

Composition of Substitutions

- The **empty substitution** $\epsilon = \{\}$ is also a substitution, and it acts as an **identity** under composition.
- Substitutions when applied to formulas are **associative**:

$$(f\theta)\delta = f(\theta\delta)$$

A **unifier** of two formulas f and g is a **substitution** δ that makes f and g syntactically identical.

Not all formulas can be unified since substitutions **only** affect **variables**.

Example:

$$P(f(x), \mathbf{a}) \qquad P(y, f(w))$$

This pair cannot be unified as there is no way of making $\mathbf{a} = f(w)$ with a substitution.

Most General Unifier (MGU)

A substitution δ of two formulas f and g is a **Most General Unifier (MGU)** if:

1. δ is a **unifier**.
2. For every other unifier θ of f and g there exist a **third substitution** λ such that

$$\theta = \delta\lambda$$

That is, every **other** unifier is **more specialized** than δ .

The **MGU** of a pair of formulas f and g is **unique** up to renaming.

The MGU is the “least specialized” way of making clauses with universal variables match.

Computing MGUs: Intuition

- We line up the two formulas and find the first **sub-expression** where they **disagree**.
- The pair of sub-expressions where they first disagree is called the **disagreement set**.
- The algorithm works by **successively fixing disagreement sets** until the two formulas become **syntactically identical**.

Most General Unifier

To find the MGU of two formulas f and g .

1. $k = 0$; $\delta_0 = \{\}$; $S_0 = \{f, g\}$.
2. REPEAT UNTIL **no more disagreement**:
3. Find disagreement set $D_k = \{e_1, e_2\}$.
4. IF $e_1 = V$, where V is a **variable**,
and $e_2 = t$, where t is a **term not containing V** ,
or vice-versa then:
 - $\delta_{k+1} = \delta_k \{V = t\}$ # **Compose** the additional substitution
 - $S_{k+1} = S_k \{V = t\}$ # **Apply** the additional substitution
 - $k = k + 1$
5. ELSE unification is **not possible**.

Consider two clauses:

$(L, Q_1, Q_2, \dots, Q_k)$

$(\neg M, R_1, R_2, \dots, R_n)$

where there exists an **MGU** δ for L and M .

We **apply** δ to both clauses, **resolve** $L\delta$ and $\neg M\delta$, and **infer** the new clause

$(Q_1\delta, Q_2\delta, \dots, Q_k\delta, R_1\delta, R_2\delta, \dots, R_n\delta)$

Resolution of Clauses with Variables: Example

$$(P(x), Q(g(x)))$$
$$(R(\mathbf{a}), Q(z), \neg P(\mathbf{a}))$$

$$L = P(x), M = P(\mathbf{a})$$
$$\delta = \{x = \mathbf{a}\}$$

$$R[1a, 2c]\{x = \mathbf{a}\}(Q(g(\mathbf{a})), R(\mathbf{a}), Q(z))$$

Resolution of Clauses with Variables: Example

$(P(x), Q(g(x)))$

$(R(\mathbf{a}), Q(z), \neg P(\mathbf{a}))$

$L = P(x), M = P(\mathbf{a})$

$\delta = \{x = \mathbf{a}\}$

$R[1a, 2c]\{x = \mathbf{a}\}(Q(g(\mathbf{a})), R(\mathbf{a}), Q(z))$

The notation is **important**.

- **R**: resolution step.
- **1a**: the **first** (**a**-th) literal in the first clause; i.e. $P(x)$.
- **2c**: the **third** (**c**-th) literal in the second clause; i.e., $\neg P(\mathbf{a})$.
 - **1a** and **2c** are the **clashing** literals.
- $\{x = \mathbf{a}\}$: the **substitution** applied to make the clashing literals identical.